

科目：データベース演習

第10回：SQL応用：関数とジョイン

講師：鈴木源吾

前回の復習

1. コマンドラインアプリケーション：データ作成
2. アプリケーション開発フレームワークとデータベース
 - アプリケーション開発フレームワークとは
 - データベース開発のフレームワーク（ORマッピング）
 - SQLAlchemyによるデータベースアクセス

今回のトピック

1. 関数で新しいカラムを作り出す
 - 関数の使い方
 - 数値計算・文字列処理
2. ジョインの基礎
 - 正規化とジョイン
 - ジョインでテーブルを連結する

topic 1

関数で新しいカラムを作り出す

SQLで関数を使う→新しいカラムを作り出す

- SQL文の中で、数値計算や文字列処理の関数を用いることができる
- プログラミングなしで、簡易にデータの加工が行えるため、非常によく使われる
- 関数を使うことは、既存のカラムから必要なカラムを計算で作り出すことになる
 - 例) 年齢カラムから、年齢層カラムを作成する (年齢: 45→年齢層: 40)
 - 例) 氏名カラムから、名字カラムを作成する (氏名: 鈴木 源吾→名字: 鈴木)
(正確に名字を取り出すのは単純な関数では難しいが・・・)

数値の計算

例題として，サンプルDBのaccess_log_wideテーブルを使用する

検索結果の数を表すsearch_hitカラムをまずは検索してみよう．下の結果が得られる
(最初の5件のみ表示．次ページ以降も同様)

```
SELECT search_hit FROM access_log_wide;
```



alt

数値の計算

（あまり意味はないが例として） search_hitカラムに1を足してみよう．以下のSQLで実行することができる → Null値をとる行を除くすべての行で数値が足される

```
SELECT search_hit + 1 FROM access_log_wide;
```

 alt

数値計算のSQLの解説



カラムとカラムの演算

前の例では、カラムに数値を加えたが、カラム同士の演算もできる
(これも意味はないが) 顧客の年齢であるcustomer_ageとsearch_hitを加えてみよう

```
SELECT customer_age + search_hit FROM access_log_wide;
```

 alt

数値計算の演算子

SQLで使える数値計算の演算子を以下に示す

- プログラミング言語の演算子の記号と基本的に同じ
- 掛け算は×ではないので注意.

 alt

SQLの除算は整数除算

- 注意！：PostgreSQLの「/」は整数どうしの割り算をするときには、割った余りを切り捨てて整数部分だけを返す
- 例：17/5の結果は3である（3.4ではない）
- 例：6/4の結果は1である
- 例：3/5の結果は0である

計算の結果だけを得たいのであれば、SELECT句だけの問合せを使うことができる

```
SELECT 17/5;
```

を実行し結果を確かめてみよう

年齢層を求めてみよう

- 「/」 演算を用いれば、年齢層を求めることができる.
- 年齢層：45歳→40代, 38歳→30代
- 年齢の10の位を取り出す
- 45歳を10で割る→結果の整数部は4→これを10倍すれば年齢層
↓顧客の氏名, 年齢, 年齢層を求めるSQL文（5行に限定）

```
SELECT customer_name, customer_age, customer_age/10*10 FROM customers LIMIT 5;
```

 alt

文字列処理：文字列の長さ

文字列処理（文字列の長さを数える，文字列をつなげる，文字列を切り出す等）は，非常によく使う処理であり，関数が用意されている

文字数を数える関数として，「char_length()」がある．これを試してみよう

- 以下のSQL文は，名前(customer_name)と名前の文字数を出している

```
SELECT customer_name, char_length(customer_name) FROM customers LIMIT 5;
```

 alt

文字列処理：文字列の連結

文字列の連結もよく使う（例：名字と名前を繋げて氏名を作る）

SQLでは、「||」という演算子を用いる

- 以下のSQL文は、顧客の年齢(customer_age)と性別(customer_gender)を連結した
- customer_ageは数値型（integer）だが、暗黙的に文字列型に変換されている

```
SELECT customer_age || customer_gender FROM customers LIMIT 5;
```

 alt

文字列処理の演算子と関数

以下に、文字列を処理するための主要な演算子と関数をまとめる（PostgreSQL）

 alt

- 注意：SQLiteでは、char_lenghtがない。lengthを使う
- 注意：DBMSによっては、文字数がバイト長になる可能性もあり

文字列処理の応用

customer_birthdayの値の年のみを求めるSQL文を考えてみよう

前のページにあったsubstringが使えるそうだ（最初の4文字をとればよい）

以下のSQL文を実行してみよう → しかしエラーとなってしまふ

- 原因は、customer_birthdayの型が文字列型ではなく日付型であること

```
SELECT substring(customer_birthday, 1, 4) FROM customers LIMIT 3;
```



alt

型のキャスト

このような場合、型を変換する必要がある、型をキャストする必要がある。 **キャスト**とは、値を別の型に変換する操作である。以下のように書く

```
cast(カラム名 AS データ型名) または カラム名::データ型名
```

例えば、customer_birthdayをtext型に変換するためには、cast(customer_birthday AS text)と書く。これを利用すれば以下の結果が得られる

```
SELECT substring(cast(customer_birthday AS text), 1, 4) FROM customers LIMIT 3;
```

 alt

例題1

customersテーブルに対して、以下のようなSQL文を作成し実行せよ
(行数が多いため、すべて結果の上限を10行に制限すること)

- (1) (すべての) customer_ageの値に100を加えた結果を求めるSQL文
- (2) customer_ageに対して、演算子%を使用して年齢層を求めるSQL文
- (3) customer_birthdayの値の日のみを求めるSQL文 (1978-08-22なら22を求める)

演習 課題5-1

itemsテーブルに対して、以下のようなSQL文を作成せよ

(行数が多いため、すべて結果の上限を10行に制限すること)

- (1) (すべての) shop_idの値に100を加えた結果を求めるSQL文
- (2) item_priceの値から、1000円未満の値を切り捨てた値を求めるSQL文
(例：item_priceが7800であれば、7000を求める、同様に13010→13000を求める)
- (3) item_nameの値の最初の5文字だけを求めるSQL文

topic 2

ジョインの基礎

正規化とジョイン

例として使っていたaccess_log_wide テーブルは、以下のように、ウェブのアクセスログとそれに伴う情報をいろいろ格納していた。そのため、customer_nameに同じ名前が複数回重複して出現するときがある（下の例では「丸山冴子」は2回出現）

alt

→ 正規化されていない

- 正規化は、データの重複を排除して、1つのテーブルには1つの事実（概念）のみ にすることだった（更新時異状をなくすのが目的）

正規化

サンプルDBに入っているaccess_logテーブルとcustomersテーブルはこれを正規化したものになっている. 顧客（customers）は単独の事実として別のテーブルに分離する

- 2つのテーブルは主キーと外部キー（customer_id）で結び付けられる

 alt

正規化とジョイン

データベースは原則正規化を行う→検索にはジョインを使うことが多くなる！

ジョイン：2つのテーブルを結合して1つの大きなテーブルを得る処理

- 結合するときに、主キーと外部キーで行を結びつける（下図のイメージ）

 alt

ジョインのSQL文

ジョインのSQL文の記述には、JOIN句を明示的に示す方法と、示さない方法がある.

例えば、このaccess_logテーブルとcustomersテーブルを、JOIN句を明示的に示さずにジョインするSQL文は以下となる

```
SELECT * FROM access_log, customers  
WHERE access_log.customer_id = customers.customer_id LIMIT 10;
```

前のページのイメージ図の点線を結びつけるのが、以下の部分である

```
access_log.customer_id = customers.customer_id
```

ジョインのSQL文：JOIN句を使う

以下のようにJOIN句を明示的に使う方法もある
こちらの方がわかりやすいかもしれない（参考書の記法）

- JOIN句で結合する2つのテーブルを示す
- ON句で結合の条件を示す
- JOINは、「INNER JOIN」（内部結合）と書いてもよい
 - 通常のジョインは内部結合. 次回に外部結合を説明予定

```
SELECT * FROM access_log
JOIN customers
ON access_log.customer_id = customers.customer_id
LIMIT 10;
```

ジョインのSQL文：テーブルの別名

ジョインでは、テーブルの名前を多く記述する必要があるが、その記述を節約するのに役立つテーブルの別名を付与する機能がある

テーブル名 AS 別名

と別名を定義すると、別名をSQL文のON句やWHERE句で使うことができる
よって、前ページのSQL文は以下のように書き換えることができる

```
SELECT * FROM access_log AS a  
JOIN customers AS c  
ON a.customer_id = c.customer_id  
LIMIT 10;
```

ジョインしてカラムを取り出す

SELECT句でカラムを指定すれば、ジョインの結果でカラムを絞って取り出すこともできる。ここで少し前に学んだ演算子も活用した例をやってみよう。

SELECT句で、request_timeと併せて、氏名（customer_name）と年齢（customer_age）を利用して、結果が、「氏名(年齢)」のように表示してみよう（このカラムをcustomer_detailとする）

```
SELECT request_time, customer_name || '(' || customer_age || ')' AS customer_detail
FROM access_log AS a
INNER JOIN customers AS c
ON a.customer_id = c.customer_id
LIMIT 10;
```

AS句はこのようなカラム名の別名として用いることもできる

前のページのSQL文の実行結果



3つのテーブルのジョイン

「誰が、いつ、いくら買っているか」1度に得るためには下図のように3つのテーブルをジョインする必要がある

 alt

3つのテーブルのジョイン

3つのテーブルをジョインするためには、JOIN句を複数並べればよい

```
SELECT * FROM orders AS o  
JOIN order_details AS d ON o.order_id = d.order_id  
JOIN customers AS c ON o.customer_id = c.customer_id  
LIMIT 10;
```

例題2

web_pagesテーブルは、ウェブページのアドレス（カラム名request_path, 例：/index.html）と人間に理解できるウェブページの名前（カラム名 page_name, 例：top）を対応づけるテーブルである。一方access_logテーブルは、ウェブページのアドレスにアクセスした時間（カラム名 request_time）や顧客ID（カラム名 customer_id）を管理する。以下のようなSQL文を作成し実行せよ

（行数が多いため、すべて結果の上限を10行に制限すること）

何時（request_time）、誰（customer_id）が、どの名前のページ(page_name)にアクセスしているかを抽出したい。上の2つのテーブルを、ウェブページのアドレス

（request_path）が等しいという条件でジョインし、前記の3つのカラムを取得するSQL文を作成せよ（カラムの並びはこの順とする）

例題2の実行結果例



演習 課題5-2

以下のようなSQL文を作成せよ

(行数が多いため、すべて結果の上限を10行に制限すること)

(4) 例題2において、topページに関する情報のみがほしい (page_nameがtop). topページに限定して、例題2の3つのカラム (request_time, customer_id, page_name) を取得するSQL文を作成せよ

(5) 例題2において、誰にあたる情報をidではなく名前 (customer_name) にしたい. 例題2の2テーブルに加え、customersテーブルを利用し、3テーブルのジョインを用いて、request_time, customer_name, page_nameの3つのカラムを取得するSQL文を作成せよ

演習 課題5-2(4)の実行結果例



演習 課題5-2(5)の実行結果例

 alt